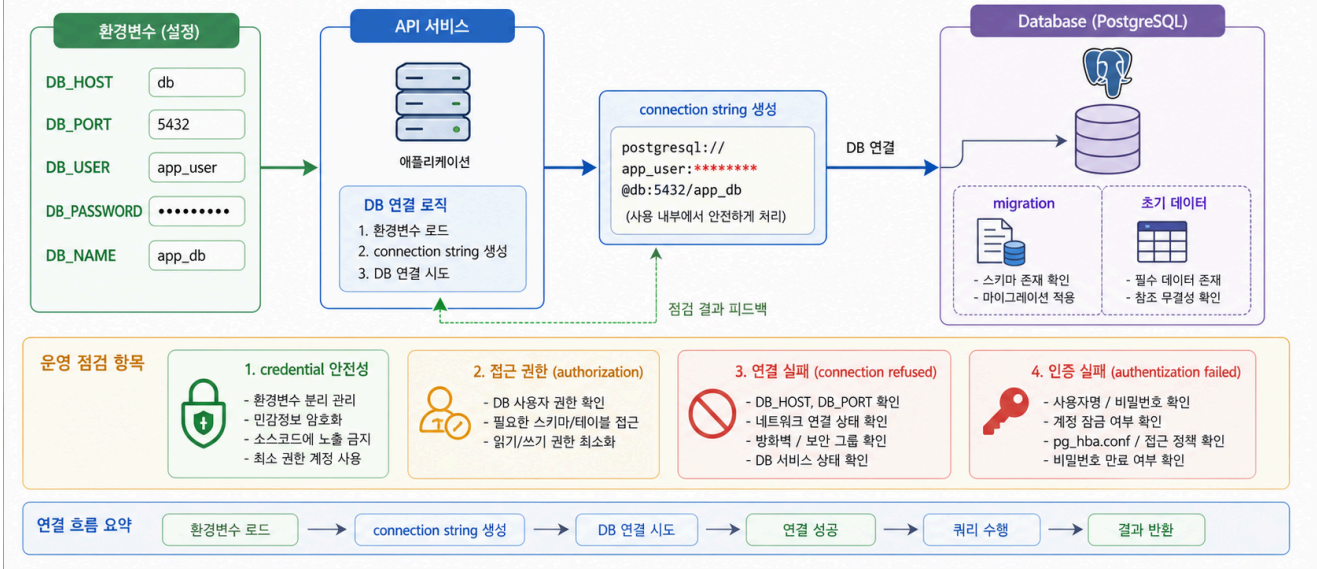


4교시: Worker Backlog와 Drain 관찰

Week 3 Day 2 Lesson 4

API와 Database 연결 운영 관점

API 서비스가 환경변수 기반 설정으로 PostgreSQL 데이터베이스에 안전하게 연결하고, 운영 시점에서 점검해야 할 항목들을 확인합니다.



수업 목표

- worker가 멈춘 동안 queue backlog가 쌓이는 상황을 재현한다.
- 단일 주문이 아니라 여러 주문을 넣어 처리량과 복구 속도를 관찰한다.
- queue length를 운영 지표로 해석한다.
- worker scale out과 중복 처리 위험을 함께 논의한다.

D1과 다른 점

D1에서는 정상 주문 하나가 처리되는 것을 봤다.

D2에서는 다음을 본다.

worker down

- > 주문 여러 개 접수
- > queue backlog 증가
- > pending orders 증가
- > worker 복구
- > queue drain
- > processed로 전환

단일 정상 처리 확인이 아니라 backlog와 recovery dynamics를 보는 것이다.

실행

```
cd week3/day2/labs/incident-scenarios
COUNT=8 ./02_backlog_drain.sh
```

COUNT를 바꿔 부하를 조절할 수 있다.

```
COUNT=20 ./02_backlog_drain.sh
```

관찰 포인트

Evidence	질문
queue length before recovery	worker 중지 중 몇 개가 쌓였는가
pending orders	업무 완료가 얼마나 밀렸는가
t+1~t+5 queue length	복구 후 얼마나 빨리 줄어드는가
processed orders	최종 처리가 완료됐는가

해석

관찰	운영 의미
queue length가 주문 수만큼 증가	worker가 유일한 소비자였다는 뜻
복구 후 0으로 감소	backlog drain 성공
일부 pending 유지	처리 실패 또는 worker 처리량 부족
worker error 반복	DB/Redis/payload 문제 가능

처리량 관점

queue length는 단순 숫자가 아니다.

```
backlog = 들어온 작업 수 - 처리된 작업 수
```

운영에서는 다음을 본다.

지표	의미
queue depth	현재 대기 중인 작업 수
drain rate	복구 후 초당 처리되는 작업 수
oldest message age	가장 오래 기다린 작업의 대기 시간
worker error rate	처리 실패 비율

현재 실습은 간단한 Redis list라서 모든 지표를 만들지는 않는다. 하지만 LLEN 변화만으로도 backlog와 drain 개념을 체감할 수 있다.

Worker scale out 질문

학생들에게 질문한다.

order-worker를 2개로 늘리면 backlog가 더 빨리 줄어들까?

가능한 실험:

```
cd week3/day1/labs/msa-demo
docker compose up -d --scale order-worker=2
docker compose ps order-worker
```

주의:

주의점	설명
worker가 늘면 처리량은 증가할 수 있다	queue 소비자가 많아진다
중복 처리 안전성이 필요하다	같은 event가 두 번 처리되면 안 된다
DB update가 idempotent해야 한다	재시도와 worker crash를 고려해야 한다
order by 처리 순서 보장은 약해질 수 있다	병렬 처리에서는 순서가 흐트러질 수 있다

Alert 기준 토론

queue length가 1이라고 무조건 장애는 아니다. 중요한 것은 지속 시간과 증가 속도다.

상황	판단
순간적으로 1~2개 쌓였다가 바로 0	정상 burst일 수 있음
계속 증가	처리량 부족 또는 worker 장애
일정 값 이상 오래 유지	alert 후보
업무 SLA 초과	사용자 영향 발생

운영 리포트 문장

order-worker 중지 중 COUNT=8 주문을 생성하자 Redis order-events queue length가 8로 증가했다.
orders table에는 동일 prefix의 주문이 pending 상태로 남았다.
order-worker 복구 후 queue length가 0으로 감소했고, 주문 상태가 processed로 변경되었다.
이는 API 접수 경로는 살아 있으나 비동기 처리 경로가 지연된 delayed failure다.

핵심 포인트

worker 장애는 "API 장애"처럼 크게 보이지 않을 수 있다. 그래서 queue backlog 지표가 필요하다.

API success + queue backlog 증가 = 업무 지연 가능성

Evidence Note

```
# W3D2S4 Backlog Drain
- count:
- queue before recovery:
- pending count:
- recovery command:
- queue after 1s:
- queue after 5s:
- final processed count:
- scaling question:
```